

# ML UNIT – 2

## 1. Define cost function in the context of Linear Regression.

1. The cost function in linear regression is a mathematical function used to measure how well the model's predictions match the actual data values. It calculates the difference between the predicted output and the real output for all training examples.
2. In simple linear regression, the hypothesis function is written as  $h_{\theta}(x) = \theta_0 + \theta_1 x$ . The cost function calculates the error between predicted values  $h_{\theta}(x)$  and actual values  $y$ .
3. The commonly used cost function for linear regression is Mean Squared Error (MSE). It is written as  $J(\theta_0, \theta_1) = \frac{1}{2m} \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2$ .
4. This function squares the errors so that negative and positive errors do not cancel each other. Larger errors are also penalized more heavily because of the square term.
5. The main goal in linear regression is to minimize the value of the cost function. When the cost function becomes small, it means the predicted values are close to the actual data values.

## 2. Describe the role of the cost function in training a model.

1. The cost function plays an important role in measuring the performance of a machine learning model during training. It tells us how far the predicted results are from the actual results.
2. During training, the model tries different parameter values to reduce prediction errors. The cost function provides a numeric value that shows whether the model is improving or not.
3. Algorithms such as **Gradient Descent** use the cost function to update model parameters. They repeatedly adjust the parameters to move toward the minimum value of the cost function.
4. The cost function guides the model to find the best-fit line that represents the relationship between variables. When the cost is minimized, the model predictions become more accurate.
5. Without a cost function, the model would not know how to improve its predictions. Therefore, the cost function acts as an **optimization target** during the training process.

### 3. Explain multivariate linear regression, its hypothesis function and cost function.

1. Multivariate linear regression is an extension of simple linear regression where multiple independent variables are used to predict a single dependent variable. It helps analyze how several factors together influence the output.
2. In this model, the hypothesis function includes many input variables. It is written as 
$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n.$$
3. Each parameter  $\theta$  represents the weight or importance of a specific feature in predicting the output. These parameters are learned from the training data during model training.
4. The cost function used in multivariate regression is similar to simple linear regression. It is written as 
$$J(\theta) = \frac{1}{2m} \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$
5. The objective of the model is to minimize this cost function using optimization methods such as gradient descent. When the cost is minimized, the model produces more accurate predictions.

### 4. Explain assumptions of linear regression.

1. One important assumption of linear regression is that there is a **linear relationship between independent and dependent variables**. This means that changes in input variables should result in proportional changes in the output variable.
2. Another assumption is that the **errors or residuals are normally distributed**. This helps in making reliable statistical inferences and accurate predictions.
3. Linear regression also assumes that the **residuals have constant variance**, which is known as homoscedasticity. If the variance changes across values, it may affect the reliability of the model.
4. The model assumes that **independent variables are not highly correlated with each other**, which is called no multicollinearity. High correlation between features can distort the effect of individual variables.
5. Another assumption is that the **observations are independent of each other**. This means the value of one data point should not influence another data point in the dataset.

**5. Explain performance metrics used for a regression: i. MAE, ii. MSE, iii. RMSE iv. R<sup>2</sup> score**

1. **Mean Absolute Error (MAE)** measures the average absolute difference between the predicted values and the actual values. It shows how far predictions are from real values without considering the direction of error.
2. **Mean Squared Error (MSE)** calculates the average of the squared differences between predicted and actual values. Squaring the error penalizes larger mistakes more heavily than smaller ones.
3. **Root Mean Squared Error (RMSE)** is the square root of the mean squared error. It gives the error value in the same unit as the target variable, which makes interpretation easier.
4. **R<sup>2</sup> Score (Coefficient of Determination)** measures how well the regression model explains the variability of the data. Its value usually ranges from 0 to 1, where a higher value indicates a better model fit.
5. These metrics help evaluate how accurately the regression model predicts outcomes. They are widely used in machine learning to compare and select the best model.

**6. Explain gradient descent algorithm.**

1. Gradient Descent is an optimization algorithm used to minimize the cost function in machine learning models. It helps find the best parameter values that reduce prediction error.
2. The algorithm starts with initial values for parameters such as  $\theta_0$  and  $\theta_1$ . These values are usually set to zero or small random numbers.
3. In each step, the algorithm updates the parameters slightly to reduce the value of the cost function. This update is based on the gradient or slope of the cost function.
4. The algorithm repeatedly adjusts the parameters in the direction where the cost decreases the most. This process continues until the cost function reaches a minimum value.
5. Gradient descent is widely used in regression and many machine learning algorithms. It is an iterative process that gradually improves the model's performance.

## 7. Analyze the impact of learning rate on Gradient Descent convergence.

1. The **learning rate ( $\alpha$ )** is a parameter that controls the size of steps taken during gradient descent. It determines how quickly the algorithm updates the model parameters.
2. If the learning rate is **too small**, the algorithm takes very tiny steps toward the minimum. As a result, the training process becomes very slow and requires many iterations.
3. If the learning rate is **too large**, the algorithm may overshoot the minimum point of the cost function. This can cause the algorithm to fail to converge properly.
4. A properly chosen learning rate allows the algorithm to reach the minimum efficiently. It helps the model converge faster while maintaining stable updates.
5. Therefore, selecting the correct learning rate is important for successful training of the model. It directly affects the speed and stability of gradient descent convergence.

## 8. Differentiate between batch gradient descent and stochastic gradient descent.

1. **Batch Gradient Descent** calculates the gradient using the entire training dataset before updating the parameters. This means the algorithm processes all examples together in each iteration.
2. **Stochastic Gradient Descent (SGD)** updates the model parameters using one training example at a time. This allows the algorithm to update parameters more frequently.
3. Batch gradient descent is more stable because it uses complete data to compute gradients. However, it can be slow when the dataset is very large.
4. Stochastic gradient descent is faster and can start improving the model immediately. But its updates may fluctuate because it uses only one example at a time.
5. In practice, stochastic gradient descent is often preferred for large datasets. Batch gradient descent is useful when the dataset is small and stable convergence is required.

## 9. Explain Logistic Regression and sigmoid function.

1. **Logistic regression** is a supervised learning algorithm mainly used for classification problems. It predicts the probability of an outcome that belongs to a particular class such as spam or not spam.
2. In logistic regression, the output variable is usually binary, meaning it can take values like **0 or 1**. For example, in medical diagnosis it can predict whether a tumor is malignant or benign.
3. Logistic regression uses a linear combination of input variables represented as  $\theta^T x$ . This value is then passed into a special function called the **sigmoid function**.
4. The **sigmoid function** converts any real number into a value between 0 and 1. Because of this property, it is useful for representing probabilities.
5. The sigmoid function is written as  $g(z) = \frac{1}{1+e^{-z}}$ . This S-shaped curve allows logistic regression to perform classification based on probability values.

## 10. Explain the cost function in logistic regression.

1. The **cost function in logistic regression** is used to measure how well the model's predictions match the actual outcomes. It calculates the error between predicted probabilities and true class labels.
2. Unlike linear regression, squared error cannot be used effectively for logistic regression. This is because it may produce incorrect optimization results for classification problems.
3. Logistic regression therefore uses a **log loss or cross-entropy cost function**. This function penalizes incorrect predictions more strongly.
4. The cost function assigns a small value when the predicted probability is close to the true label. If the prediction is far from the correct class, the cost becomes large.
5. During training, algorithms such as gradient descent minimize this cost function. When the cost is minimized, the model becomes better at classifying data correctly.

## 11. Explain how predictions are done in logistic regression.

1. In logistic regression, predictions are made by first calculating the value  $\theta^T x$  using the model parameters and input features. This value represents the weighted sum of the input variables.
2. The result is then passed through the **sigmoid function**, which converts it into a probability value between 0 and 1. This probability represents the likelihood that the output belongs to class 1.
3. For example, if the model outputs a value of **0.7**, it means there is a 70% probability that the output belongs to class 1. The remaining 30% probability corresponds to class 0.
4. A **decision boundary** is used to convert probabilities into class labels. Usually, if the probability is greater than or equal to **0.5**, the output is predicted as class 1.
5. If the probability is less than 0.5, the model predicts class 0. This method allows logistic regression to perform classification based on probability thresholds.

## 12. What is the difference between precision and recall? How does F1 combine both?

1. **Precision** measures how many of the predicted positive results are actually correct. It focuses on the accuracy of positive predictions made by the model.
2. **Recall** measures how many of the actual positive cases were correctly identified by the model. It focuses on the model's ability to detect all relevant positive instances.
3. Precision is important when the cost of false positives is high, such as spam detection. Recall is important when missing a positive case is critical, such as disease detection.
4. The **F1 score** is a metric that combines both precision and recall into a single value. It is calculated as the harmonic mean of precision and recall.
5. The F1 score provides a balanced evaluation of the model's performance. It is especially useful when the dataset is imbalanced and both precision and recall are important.

### 13. Explain Confusion Matrix with the help of an example.

1. A **confusion matrix** is a table used to evaluate the performance of a classification model. It compares the predicted values of the model with the actual values from the dataset.
2. The matrix usually contains four components: **True Positive (TP)**, **True Negative (TN)**, **False Positive (FP)**, and **False Negative (FN)**. These values help understand how correctly the model classifies data.
3. **True Positive (TP)** means the model predicted positive and the actual value was also positive. **True Negative (TN)** means the model predicted negative and the actual value was also negative.
4. **False Positive (FP)** occurs when the model predicts positive but the actual value is negative. **False Negative (FN)** occurs when the model predicts negative but the actual value is positive.
5. For example, in a spam email classifier, if the model correctly identifies a spam email it is a TP. If it incorrectly labels a normal email as spam, it is an FP, which shows an error in prediction.

### 14. What is overfitting in machine learning, causes of overfitting, and how can it be avoided?

1. **Overfitting** occurs when a machine learning model learns the training data too well, including noise and unnecessary details. As a result, the model performs well on training data but poorly on new or unseen data.
2. One major cause of overfitting is using a **very complex model with too many parameters**. When the model becomes too complex, it starts memorizing the data instead of learning patterns.
3. Another cause is **insufficient training data or noisy data**. If the dataset is small or contains many errors, the model may not generalize properly.
4. Overfitting can be avoided using techniques such as **regularization methods like Ridge and Lasso**. These techniques add penalties to large model parameters and reduce complexity.
5. Other solutions include **using more training data, cross-validation, and simplifying the model**. These methods help the model learn general patterns instead of memorizing the dataset.

**15. Explain the following performance metrics for classification: Accuracy, Confusion Matrix, Precision, Recall, F1 Score, ROC Curve.**

1. **Accuracy** measures the overall correctness of a classification model. It is calculated as the ratio of correct predictions to the total number of predictions made.
2. A **Confusion Matrix** is a table that shows the number of correct and incorrect predictions made by the model. It includes values such as true positives, true negatives, false positives, and false negatives.
3. **Precision** measures the proportion of predicted positive cases that are actually positive. It helps evaluate how accurate the model is when predicting positive outcomes.
4. **Recall** measures how many actual positive cases are correctly identified by the model. It is important when missing positive cases could be harmful, such as in disease detection.
5. The **F1 Score** combines precision and recall using their harmonic mean to give a balanced performance measure. The **ROC Curve** is a graphical representation that shows the trade-off between true positive rate and false positive rate for different thresholds.

**16. What is underfitting in machine learning and how can it be avoided?**

1. **Underfitting** occurs when a machine learning model is too simple to capture the underlying pattern in the data. As a result, the model performs poorly on both training data and testing data.
2. This problem usually happens when the model has **too few features or insufficient complexity**. A simple model cannot learn the relationships between variables properly.
3. Underfitting can also occur when the model is **trained for too few iterations or with poor parameter settings**. In such cases, the model stops learning before reaching good performance.
4. One way to avoid underfitting is to **increase the complexity of the model** by adding more features or using more advanced algorithms. This allows the model to learn more detailed patterns in the data.
5. Another solution is to **train the model for more time and tune its parameters properly**. Proper training and feature engineering can significantly improve the model's ability to fit the data.



## 17. Design a strategy to reduce overfitting in a machine learning model.

1. One effective strategy to reduce overfitting is **increasing the size of the training dataset**. When more data is used, the model learns general patterns instead of memorizing specific examples.
2. Another method is to **simplify the model by reducing the number of features or parameters**. A simpler model is less likely to capture noise present in the training data.
3. **Regularization techniques such as Ridge and Lasso regression** can also be used to reduce overfitting. These methods add a penalty to large parameter values and prevent the model from becoming too complex.
4. **Cross-validation techniques** can help evaluate model performance on different subsets of data. This helps ensure that the model performs consistently across different data samples.
5. Another strategy is to use **feature selection and remove irrelevant variables**. Removing unnecessary features helps the model focus only on important patterns in the dataset.

## 18. Analyze a model to identify overfitting and underfitting.

1. To analyze a model, we first compare **training error and testing error**. These errors help us understand how well the model performs on known data and unseen data.
2. **Overfitting occurs when the training error is very low but the testing error is high**. This means the model memorized the training data but cannot generalize to new data.
3. **Underfitting occurs when both training error and testing error are high**. In this case, the model is too simple to capture patterns in the data.
4. Learning curves can also help analyze these problems. They show how training and validation errors change as the model learns.
5. By analyzing these errors and curves, we can decide whether to increase model complexity or simplify the model. This helps improve overall model performance.

### 19. Explain trade-off between bias and variance.

1. The **bias-variance trade-off** describes the balance between model simplicity and model complexity. It explains how different models perform on training data and unseen data.
2. **Bias** refers to the error caused by overly simple assumptions in the learning algorithm. High bias usually leads to underfitting because the model cannot capture the true relationship in the data.
3. **Variance** refers to how much the model's predictions change when the training data changes. High variance usually leads to overfitting because the model becomes too sensitive to training data.
4. A good machine learning model should maintain a balance between bias and variance. This balance helps the model perform well on both training data and new data.
5. The main goal in machine learning is to find the **optimal point where both bias and variance are reasonably low**. Achieving this balance improves prediction accuracy and generalization ability.

### 20. Differentiate between training error and generalization error.

| Training Error                                                                                                                                                                                              | Generalization Error                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Training error is the error calculated using the same dataset that was used to train the model. It shows how well the model has learned the patterns present in the training data.                          | Generalization error is the error measured on new and unseen data that was not used during model training. It indicates how well the model performs in real-world situations. |
| Training error is usually lower because the model has already seen the training data during the learning process.                                                                                           | Generalization error is usually higher because the model must make predictions on unfamiliar data.                                                                            |
| If the training error is very low but the testing or validation error is high, it usually indicates overfitting. In this situation the model memorizes training data but fails to perform well on new data. | If both training error and testing error are high, the model is likely underfitting. This means the model is too simple to capture the underlying patterns in the dataset.    |
| Training error alone cannot fully measure the performance of a machine learning model. A model may appear accurate on training data but still fail on unseen data.                                          | Generalization error is considered a better indicator of the model's real performance. Good machine learning models aim to minimize this error to make reliable predictions.  |

## 21. Explain validation error.

1. **Validation error** is the error measured on a validation dataset that is separate from the training dataset. It helps evaluate how well a machine learning model performs on unseen data during the training process.
2. The validation dataset is not used to train the model parameters. Instead, it is used to check the model's performance while tuning hyperparameters.
3. Validation error helps detect problems such as **overfitting and underfitting** in a model. If validation error is much higher than training error, the model may be overfitting.
4. During model development, different models or parameter settings are compared using validation error. The model with the lowest validation error is usually selected.
5. Therefore, validation error plays an important role in improving model performance. It ensures that the model generalizes well to new and unseen data.

## 22. Explain regularization in Machine Learning.

1. **Regularization** is a technique used in machine learning to reduce overfitting in models. It works by adding a penalty term to the cost function during training.
2. The purpose of regularization is to prevent the model from becoming too complex. By limiting large parameter values, the model focuses on important patterns instead of noise.
3. Regularization helps improve the model's ability to **generalize to new data**. This means the model performs well not only on training data but also on unseen data.
4. Two common types of regularization are **L1 regularization (Lasso)** and **L2 regularization (Ridge)**. Both methods add different types of penalty terms to the cost function.
5. By controlling model complexity, regularization creates a balance between bias and variance. This leads to more stable and reliable machine learning models.

### 23. Explain L1 regularization.

1. **L1 regularization**, also known as **Lasso regression**, is a technique used to reduce model complexity. It adds the absolute value of the coefficients as a penalty term to the cost function.
2. The penalty term encourages the model to reduce the magnitude of coefficients. As a result, some coefficients may become exactly zero.
3. When coefficients become zero, the corresponding features are removed from the model. This process is known as **feature selection**.
4. L1 regularization is useful when dealing with datasets that contain many irrelevant features. It helps identify the most important features that influence the output.
5. By removing unnecessary features, L1 regularization simplifies the model and reduces overfitting. This improves the model's prediction performance.

### 24. Explain L2 regularization.

1. **L2 regularization**, also called **Ridge regression**, is another technique used to reduce overfitting in machine learning models. It adds the square of the coefficients as a penalty term to the cost function.
2. Unlike L1 regularization, L2 does not force coefficients to become exactly zero. Instead, it reduces the magnitude of coefficients gradually.
3. This method keeps all features in the model but limits their influence. As a result, the model becomes more stable and less sensitive to noise.
4. L2 regularization is commonly used when all features are considered important. It helps prevent extremely large coefficient values in the model.
5. By controlling the size of parameters, L2 regularization reduces model complexity and improves generalization performance.

## 25. Differentiate between L1 and L2 regularization.

| Basis                  | L1 Regularization (Lasso)                                                                                                                 | L2 Regularization (Ridge)                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Penalty Term           | L1 regularization adds the absolute value of the coefficients as a penalty term to the cost function.                                     | L2 regularization adds the square of the coefficients as a penalty term to the cost function.                                                 |
| Effect on Coefficients | In L1 regularization, some coefficients can become exactly zero during training. This removes the influence of those features completely. | In L2 regularization, coefficients become smaller but rarely reach exactly zero. All features remain in the model but with reduced influence. |
| Feature Selection      | L1 regularization can perform automatic feature selection by eliminating unnecessary variables                                            | L2 regularization does not remove features from the model. Instead, it keeps all features and distributes weights among them.                 |
| When to Use            | L1 regularization is useful when the dataset contains many irrelevant or redundant features.                                              | L2 regularization is useful when most features are relevant and should be retained.                                                           |
| Impact on Model        | L1 regularization helps reduce overfitting by simplifying the model and removing unnecessary features.                                    | L2 regularization also reduces overfitting by shrinking coefficient values.                                                                   |

## 26. Explain the impact of regularization parameter on the model.

1. The **regularization parameter ( $\lambda$ )** controls the strength of the penalty added to the cost function. It determines how much the model complexity should be reduced.
2. When the regularization parameter is **very small**, the penalty effect is minimal. In this case, the model behaves almost like a normal regression model without regularization.
3. When the regularization parameter is **very large**, the penalty becomes strong. This forces the coefficients to shrink significantly and may simplify the model too much.
4. If the parameter is too large, the model may become too simple and cause **underfitting**. Therefore, choosing the correct value of  $\lambda$  is important.
5. Proper tuning of the regularization parameter helps achieve a balance between bias and variance. This leads to better generalization and improved model performance.

## 27. Analyze the cases when L1 regularization should be used.

1. L1 regularization should be used when the dataset contains **a large number of features**. It helps remove unnecessary features by setting some coefficients to zero.
2. It is useful when many features in the dataset are **irrelevant or redundant**. By removing such features, the model becomes simpler and easier to interpret.
3. L1 regularization is also helpful in **high-dimensional datasets**, where the number of features is very large compared to the number of observations. This situation is common in text analysis and genomic data.
4. When feature selection is important for understanding the model, L1 regularization is preferred. It highlights only the most significant variables that influence the output.
5. Therefore, L1 regularization is suitable for problems where reducing the number of features improves efficiency and interpretability.

## 28. Analyze the cases when L2 regularization should be used.

1. L2 regularization should be used when **all features in the dataset are important** and should remain in the model. Instead of removing features, it reduces their influence by shrinking coefficients.
2. It is useful when the dataset contains **many correlated features**. L2 regularization distributes the weights among correlated variables instead of eliminating them.
3. L2 regularization works well when we want a **stable and smooth model**. It prevents extremely large coefficient values that can cause unstable predictions.
4. This technique is often used in regression models where the goal is to improve prediction accuracy rather than feature selection. It helps the model generalize better to unseen data.
5. Therefore, L2 regularization is suitable when we want to reduce overfitting while keeping all features in the model. It ensures the model remains balanced and reliable.

**29. Write a program to implement Linear Regression on a dataset and evaluate the model using appropriate performance metrics.**

1. First, import required libraries such as **pandas**, **numpy**, and **sklearn** to handle the dataset and implement the regression model. Then load the dataset and divide it into **independent variables (X)** and **dependent variable (y)**.
2. Next, split the dataset into **training and testing sets** using `train_test_split` so that the model can be trained and evaluated properly. After splitting, create the **Linear Regression model** using `LinearRegression()` from `sklearn`.
3. Train the model using the training dataset by calling the `fit()` function. After training, use the `predict()` function to generate predictions for the test dataset.
4. Evaluate the model using regression performance metrics such as **MAE**, **MSE**, **RMSE**, and **R<sup>2</sup> score**. These metrics help measure how close the predicted values are to the actual values.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {
    "Area": [1000, 1500, 1800, 2400, 3000],
    "Price": [200000, 250000, 280000, 350000, 400000]
}

df = pd.DataFrame(data)

X = df[["Area"]]
y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))
```

**30. Write a program to implement Logistic Regression on a dataset and evaluate the model using appropriate performance metrics.**

1. Logistic regression is used for **classification problems where the output is categorical**, such as 0 or 1. It predicts the probability of a class using the **sigmoid function**.
2. First, import libraries such as **pandas, sklearn, and numpy**, and load the dataset. Then separate the dataset into **features (X)** and **target variable (y)**.
3. Split the dataset into **training and testing sets** using `train_test_split`. After that, create a logistic regression model using `LogisticRegression()`.
4. Train the model using the training dataset and generate predictions using the testing dataset. This allows the model to classify the data into different categories.
5. Evaluate the model using classification performance metrics such as **Accuracy, Precision, Recall, F1 Score, and Confusion Matrix**.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Pass": [0, 0, 0, 0, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[["Hours"]]
y = df["Pass"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```



